

دليل عرض وتشغيل مشروع جدولة المعالج

مؤمن عبدالله الشامي

مقرر هياكل البيانات والخوارزميات نظري

إشراف الدكتور بلال مرشد

يوضح هذا الدليل كيفية تشغيل المشروع أمام الدكتور، وترتيب الحديث، وقراءة النتائج، والوصول إلى الأجزاء المهمة من الكود. يعتمد العرض على العينة الموجودة فعلاً داخل البرنامج؛ ولا يحتاج إلى إدخال عمليات أو أرقام عند التشغيل.

مسار العرض المختصر

افتح الكود أولاً، ثم شغل البرنامج، واعرض البيانات والخوارزميات والمقارنة. انتقل بعد ذلك إلى مواضيع هياكل البيانات والمعادلات. مدة العرض المقترحة ست دقائق، ثم تترك بقية الوقت لأسئلة الدكتور.

الزمن	ما يظهر على الشاشة	الفكرة التي تشرحها
٠٠ إلى ٠١ دقيقة	الكود والعينة داخل main	هدف المشروع وبيانات العمليات
٠١ إلى ٠٢ دقيقة	نتيجة التشغيل وجدول العمليات	تشغيل تلقائي ومعاني الأعمدة
٠٢ إلى ٠٣ دقيقة	ترتيب التنفيذ ومخطط Gantt	اختلاف اختيار العملية التالية
٠٣ إلى ٠٤ دقيقة	جدول PERFORMANCE COMPARISON	مقارنة WT و TAT واستغلال المعالج
٠٤ إلى ٠٥ دقيقة	هياكل البيانات ودوال الجدولة	لماذا اختير كل هيكل
٠٥ إلى ٠٦ دقيقة	دالة الإحصاءات ورابط GitHub	مثال حساب وحدود المحاكاة

جهز هذه الأشياء قبل المناقشة

١. فك ضغط الحزمة في مجلد واضح مثل C:\CPU_Project ، ثم جرّب التجميع والتشغيل على الجهاز نفسه قبل الموعد.
٢. افتح الملف المصدر والتقرير الرسمي ونتيجة تشغيل حديثة. كبر الخط بالقدر الذي يجعل الأعمدة واضحة على الشاشة.
٣. بعد تجهيز المترجم، يعمل البرنامج دون إنترنت. احتفظ بنسخة محلية من الملفات.
٤. تدرب على شرح فرق SJF و Priority ، وحساب نتيجة عملية واحدة، وسبب ظهور 91.67% في الخوارزميات الأربع.

تشغيل المشروع أمام الدكتور

الطريقة الأولى من خلال Dev C++

١. افتح Dev-C++ المثبت على الجهاز، ثم اختر File ثم Open.

٢. من مجلد المشروع افتح الملف الآتي:

src\CPU_Scheduling_Simulator.cpp

٣. تأكد أن المترجم يدعم معيار C++17. في إصدارات Dev-C++ التي توفر الخيار، افتح Tools ثم Compiler Options وابحث عن Language standard ، أو أضف الخيار التالي إلى خيارات المترجم:

-std=c++17

٤. اختر Compile & Run. في كثير من الإصدارات يكون الاختصار F11 ، وقد يختلف حسب الإصدار.

٥. تظهر نتائج الخوارزميات الأربع مباشرة. لا تكتب 2 أو 9 ولا بيانات عمليات؛ هذه النسخة تعمل تلقائيًا.

إذا أغلقت نافذة النتائج بعد انتهاء التنفيذ، استخدم ملف العرض أدناه أو شغل الملف التنفيذي من نافذة Command Prompt مفتوحة.

الطريقة الثانية ملف العرض المرفق

إذا كان الأمر g++ معروفًا في PATH ، انقر مرتين على الملف الموجود في جذر الحزمة:

START_DEMO_WINDOWS.bat

يجمع الملف المصدر الحالي، ويشغل البرنامج، ويحفظ ناتج هذا التشغيل، ثم يفتحه في المفكرة. يوقف العملية عند فشل التجميع أو التشغيل، حتى لا يعرض نتيجة قديمة على أنها جديدة.

output\Live_Output.txt

في المفكرة، انتقل إلى بداية الملف لعرض العينة. ثم استخدم Ctrl+F للبحث عن FCFS أو SJF أو PERFORMANCE COMPARISON. يفضل خط أحادي المسافة مثل Consolas وتعطيل التفاف الكلمات إذا اختلت محاذاة الأعمدة.

إذا ظهر أن g++ غير موجود

قد يكون Dev-C++ قادرًا على التجميع داخل البرنامج، بينما الأمر g++ غير متاح في PATH. عندها استخدم الطريقة الأولى. وإذا أردت تشغيل ملف العرض، أضف مجلد bin الخاص بالمترجم إلى PATH ثم أعد فتح نافذة الأوامر وتحقق بالأمر:

g++ --version

وجود محرر الكود وحده لا يكفي؛ يجب أن يتوافر مترجم C++17. ملف التشغيل المرفق لا يثبت المترجم تلقائيًا.

الكلام المقترح أثناء العرض

البداية وهدف المشروع

«السلام عليكم دكتور. هذا مشروع محاكاة لجدولة المعالج بلغة ++C، ويقارن بين أربع خوارزميات باستخدام نفس بيانات العمليات. المقارنة تعتمد على زمن الانتظار وزمن الدوران ونسبة استغلال المعالج، مع استخدام هياكل بيانات مناسبة لكل خوارزمية.»

عند فتح العينة وتشغيل البرنامج

«بيانات العمليات موجودة في المصفوفة sample داخل main. لكل عملية معرف ووقت وصول وزمن تنفيذ وأولوية. العينة مدمجة لتسهيل التشغيل وتكرار المقارنة بنفس البيانات، وقيمة الشريحة الزمنية في Round Robin تساوي وحدتين.»

الإجراء على الجهاز: أشر إلى sample و timeQuantum ، ثم نفذ Compile & Run أو شغل START_DEMO_WINDOWS.bat.

عند شرح ترتيب التنفيذ

«FCFS يخدم العمليات حسب ترتيب وصولها. SJF يختار أقصر زمن تنفيذ من العمليات التي وصلت. Priority يختار أعلى أولوية، والرقم الأصغر هنا يعني أولوية أعلى. Round Robin يوزع المعالج على العمليات في شرائح زمنية ويعيد العملية غير المنتهية إلى آخر الطابور.»

«يبدأ المعالج بخمول من صفر إلى اثنين، لأن أول عملية تصل عند الزمن اثنين. و P1 تبدأ أولاً في الخوارزميات الأربع لأنها العملية الوحيدة الموجودة وقتها.»

عند الوصول إلى المقارنة النهائية

«في هذه العينة، SJF أعطت أقل متوسط انتظار وهو 5.00 ، وأقل متوسط دوران وهو 9.40. النتيجة مرتبطة بالعينة والافتراضات. Round Robin تقدم تناوبًا بين العمليات، لكنها لم تعط أقل انتظار في هذه التجربة.»

«نسبة استغلال المعالج متساوية هنا لأن إجمالي زمن التنفيذ 22 ، والخمول وحدتان فقط، والزمن الكلي 24 ، وتكلفة تبديل السياق مفترضة بصفر.»

عند فتح هياكل البيانات والختام

«الطابور الدائري مستخدم في FCFS و Round Robin. و Min-Heap مستخدمة كطابور أولوية في SJF و Priority. القائمة المرتبطة تحفظ العمليات الأصلية، والمصفوفات تحفظ البيانات والنتائج، والمكدس يعرض سجل التنفيذ بالعكس.»

«ملفات المشروع والتقرير ودليل التشغيل موجودة في المستودع. ويمكن تطوير المحاكاة لاحقًا بإضافة تكلفة تبديل السياق وخوارزميات أخرى.»

[فتح مستودع المشروع على GitHub](#)

قراءة العينة والنتائج بسهولة

الزمن في المحاكاة وحدات افتراضية متسقة، وليس زمن التنفيذ الحقيقي الذي يقيسه الجهاز. الرقم الأصغر في عمود الأولوية يعني أولوية أعلى.

العملية	الوصول AT	التنفيذ BT	الأولوية
P1	2	5	3
P2	3	2	1
P3	4	8	4
P4	5	3	2
P5	7	4	1

ترتيب التنفيذ في الخوارزميات الثلاث غير المستبقة

الخوارزمية	ترتيب التنفيذ
FCFS	P1 → P2 → P3 → P4 → P5
SJF	P1 → P2 → P4 → P5 → P3
Priority	P1 → P2 → P5 → P4 → P3

بعد انتهاء P2 عند الزمن 9 يظهر الفرق بوضوح: FCFS تختار P3 لأنها الأسبق وصولاً بين المنتظرين، و SJF تختار P4 لأن زمن تنفيذها 3 وهو الأقصر، و Priority تختار P5 لأن أولويتها 1 وهي الأعلى.

مثال سريع على Round Robin

تعمل P1 من 2 إلى 4 ويتبقى لها 3. تعمل P2 من 4 إلى 6 وتنتهي، ثم P3 من 6 إلى 8. تتكرر بعض أسماء العمليات لأنها تعود إلى الطابور حتى ينتهي زمنها المتبقي.

الأرقام التي تحتاج شرحها

الخوارزمية	متوسط WT	متوسط TAT	استغلال المعالج
FCFS	6.80	11.20	91.67%
SJF	5.00	9.40	91.67%
Priority	5.20	9.60	91.67%
RR q=2	8.40	12.80	91.67%

تستخدم المقارنة بيانات العينة نفسها في الخوارزميات الأربع. يفسر جدول النتائج الاختلاف في ترتيب الخدمة، ولا يكفي وحده للحكم أن خوارزمية واحدة أفضل في كل الحالات.

شرح الحسابات بمثال واحد

الاختصار	معناه
AT	وقت وصول العملية إلى النظام
BT	زمن التنفيذ الذي تحتاجه العملية
CT	وقت اكتمال تنفيذ العملية
TAT	المدة من الوصول حتى الاكتمال
WT	مجموع الوقت الذي انتظرت فيه العملية دون تنفيذ

المعادلات المستخدمة في الكود

$$TAT = CT - AT$$

$$WT = TAT - BT$$

مثال P2 في خوارزمية FCFS وصلت عند 3 ، وتحتاج وحدتين للتنفيذ، واكتملت عند 9.

$$TAT = 9 - 3 = 6$$

$$WT = 6 - 2 = 4$$

التفسير الشفهي: «قضت P2 ست وحدات داخل النظام، منها وحدتان تنفيذ وأربع وحدات انتظار».

لماذا لا نحسب انتظار Round Robin من أول بدء فقط

قد تنتظر العملية أكثر من مرة بين الشرائح الزمنية. لذلك يحسب البرنامج مجموع الانتظار من وقت الاكتمال، ثم يطرح الوصول وزمن التنفيذ الكلي. مثلاً P1 في Round Robin اكتملت عند 17 ، ووصلت عند 2 ، وزمن تنفيذها 5:

$$TAT = 17 - 2 = 15$$

$$WT = 15 - 5 = 10$$

حساب استغلال المعالج

نجمع أزمدة التنفيذ للعمليات الخمس، فيكون مجموعها 22 وحدة زمنية. تبدأ المحاكاة من الزمن صفر وتنتهي عند 24 ، وفي البداية توجد وحدتا خمول.

$$Busy = 5 + 2 + 8 + 3 + 4 = 22$$

$$Total = 24 \quad Idle = 24 - 22 = 2$$

$$CPU \text{ Utilization} = (22 / 24) * 100 = 91.67\%$$

تساوي النسبة صحيح لهذه العينة، لأن جميع الخوارزميات لها مقدار العمل والخمول نفسه. لو احُسبت تكلفة تبديل السياق، فقد تختلف النتائج. النسبة هنا مؤشر محاكاة، وليست قراءة مدير المهام في Windows.

أين تفتح الكود أثناء المناقشة

استخدم البحث في محرر الكود عن الأسماء التالية. اشرح وظيفة الجزء وعلاقته بالمشروع، ثم انتقل إلى الجزء التالي دون قراءة الملف كاملاً سطرًا سطرًا.

موضع البحث	ما تشرحه
main	إنشاء العينة، تجهيز القائمة والمصفوفة، تشغيل الخوارزميات ثم عرض النتائج
Process	بيانات العملية: المعرف والوصول والتنفيذ والأولوية
runFCFS	سحب العملية من مقدمة الطابور وتنفيذها حتى تنتهي
runSJF / runPriority	اختيار العملية من Min-Heap وفق زمن التنفيذ أو الأولوية
runRoundRobin	تحديث الزمن المتبقي وإعادة العملية إلى الطابور إذا لم تنته
calculateFinalStatistics	حساب WT و TAT ومتوسطاتهما واستغلال المعالج
printComparison	طباعة المقارنة واختيار أقل متوسط انتظار ودوران

أسطر مهمة في Round Robin

```
int runTime = min(timeQuantum, remainingTime[index]);
remainingTime[index] -= runTime;
```

مدة التشغيل هي الأصغر بين الشريحة الزمنية والزمن المتبقي. إذا بقي وقت للعملية، يعيدها البرنامج إلى آخر الطابور. العمليات التي وصلت أثناء الشريحة تدخل الطابور قبل إعادة العملية الحالية.

ربط هياكل البيانات بوظائفها

اسم الهيكل في الكود	سبب استخدامه
CircularQueue	ترتيب FIFO في FCFS ، وتدوير العمليات في RR
MinHeapPriorityQueue	اختيار أصغر Burst أو أصغر رقم أولوية
ProcessLinkedList	إضافة العمليات الأصلية كعقد مترابطة ثم نسخها إلى مصفوفة
SegmentStack	عرض سجل التنفيذ من الأحدث إلى الأقدم وفق LIFO
Array	حفظ العمليات والنتائج والزمن المتبقي ومقاطع Gantt

وجود Stack هنا لغرض عرض السجل العكسي، ولا يغير قرار الجدولة. والـ Min-Heap طابور أولوية له معيار مقارنة، وليست طابور FIFO عاديًا.

أسئلة متوقعة عن الخوارزميات

ما الفرق بين FCFS و SJF ؟

FCFS تختار الأسبق وصولاً. SJF تختار أقصر زمن تنفيذ بين العمليات التي وصلت ولم تنته. لا يجوز اختيار عملية قبل وقت وصولها.

ما معنى Non Preemptive ؟

بعد بدء العملية تستمر حتى تنتهي. نسختا SJF و Priority في هذا المشروع غير مستبقتين؛ أما RR فتوقف العملية مؤقتاً عند انتهاء الشريحة إذا بقي لها وقت.

لماذا بدأت P1 أولاً رغم أن لها أولوية أقل وزمناً أطول من P2 ؟

عند الزمن 2 لم تصل إلا P1. وصلت P2 عند 3 ، والخوارزميات غير المستبقة لا توقف P1 لمجرد وصول عملية جديدة.

هل الرقم الأكبر يعني أولوية أعلى؟

ليس في هذا البرنامج. القاعدة المحددة هنا: الرقم الأصغر أعلى أولوية. لذلك أولوية 1 أعلى من أولوية 3.

كيف يعالج البرنامج تساوي زمن التنفيذ أو الأولوية؟

يستخدم وقت الوصول لكسر التعادل، ثم ترتيب العملية في المصفوفة إذا تساوى وقت الوصول أيضاً. في FCFS يحافظ ترتيب الوصول المتساوي على ترتيب المصفوفة.

لماذا تظهر بعض العمليات أكثر من مرة في Round Robin ؟

لأنها تحتاج زمناً أكبر من الشريحة؛ تنفذ جزءاً، ثم تعود إلى الطابور حتى تكمل زمنها المتبقي. وقد تظهر العملية نفسها مرتين متتاليتين إذا لم توجد عملية أخرى جاهزة.

ماذا يحدث إذا كبرت الشريحة الزمنية كثيراً؟

إذا أصبحت أكبر من أو مساوية لأزمة التنفيذ، تستطيع كل عملية الإكمال عند أول دور؛ فيقترب السلوك من FCFS. أما تصغيرها فيزيد عدد الشرائح، بينما هذا المشروع لا يحتسب تكلفة تبديل السياق.

هل SJF هي الأفضل دائماً؟

هي الأفضل من حيث المتوسطات المعروضة في هذه العينة. لا نستنتج من تجربة واحدة أنها الأفضل في كل وضع؛ قد يكون الهدف سرعة الاستجابة أو العدالة أو احترام الأولوية.

أسئلة متوقعة عن الكود والتشغيل

ما الفرق بين Stack و Queue ؟

Queue تعمل بمبدأ FIFO: أول داخل أول خارج. Stack تعمل بمبدأ LIFO: آخر داخل أول خارج. لذلك الطابور مناسب لترتيب الخدمة، والمكدس مناسب لعرض السجل بالعكس.

لماذا استخدمت Min Heap في SJF و Priority ؟

لأن أصغر مفتاح يبقى في القمة. في SJF المفتاح هو زمن التنفيذ، وفي Priority المفتاح هو رقم الأولوية. الإدراج والإزالة فيهما من رتبة $O(\log n)$.

أين توجد بيانات العمليات وكيف يمكن تغييرها؟

داخل sample في main. يغير الطالب قيم Arrival و Burst و Priority، ويحفظ الملف ثم يعيد التجميع والتشغيل. النسخة الحالية لا تتلقى هذه البيانات من لوحة المفاتيح.

هل هذه محاكاة لنظام تشغيل حقيقي؟

هي محاكاة تعليمية لمعالج واحد ودفعة CPU لكل عملية. تفترض معرفة زمن التنفيذ مسبقاً، وتتجاهل زمن تبديل السياق وعمليات الإدخال والإخراج. لا تتحكم في جدولة عمليات Windows الفعلية.

هل يدعم الكود أي عدد وأي قيم للعمليات؟

الحد المحدد للمصفوفات 20 عملية، وحد مقاطع Gantt هو 1000. العينة الحالية صحيحة. عند تحويله إلى إدخال عام يجب إضافة تحقق شامل من العدد والقيم؛ أزمدة التنفيذ موجبة، والوصول غير سالب، والشريحة موجبة.

ما الفرق بين المخرجات المحفوظة والتشغيل الحي؟

Sample_Output.txt نسخة مرجعية محفوظة. ملف START_DEMO_WINDOWS.bat يعيد تجميع الكود وتشغيله، ثم ينشئ Live_Output.txt من هذا التشغيل. أي تغيير في العينة يستلزم إعادة التجميع لتظهر نتيجته.

إذا تعطل العرض

فشل التجميع: اقرأ أول رسالة خطأ وتحقق من C++17 والمسار. نتائج قديمة: أعد التجميع وشغل الملف التنفيذي الجديد. تعذر الاتصال: اعرض النسخة المحلية. إذا استخدمت Sample_Output.txt كمرجع، وضح أنه ناتج محفوظ، ثم عد إلى التشغيل الفعلي عند توفره.

مراجعة أخيرة قبل الدخول

تأكد أنك تستطيع تشغيل البرنامج، وشرح العينة، وتمييز الخوارزميات الأربع، وحساب مثال P2، وشرح 91.67%، وتحديد موضع كل هيكل بيانات. إذا سألك الدكتور عن جزء لا تتذكره، افتح موضعه في الكود وتتبع قيمه خطوة بخطوة.